

# **FastAPI**

## **A Complete Beginner's Guide to APIs and FastAPI**

Explained from zero, with simple real-world analogies and Python examples

Purpose of this book: A reusable reference you can return to whenever you need to understand APIs, FastAPI, and how FastAPI connects Python applications to databases, AI/ML models, and LLM-based systems.

# Table of Contents

1. What is an API?
2. A real-world example
3. Another example: Google Maps
4. API in very simple words
5. Why do we need APIs?
6. API is NOT a programming language
7. So what is FastAPI?
8. Python without FastAPI
9. Turning Python into an API
10. Let's understand the code
11. What is /add?
12. What is an endpoint?
13. GET, POST, PUT and DELETE
14. A simple analogy
15. What does an API return?
16. A complete FastAPI example
17. How does this actually work?
18. What is 127.0.0.1?
19. What is port 8000?
20. Installing FastAPI
21. Create a project
22. Create a virtual environment
23. Activate it
24. Install FastAPI
25. Create your first application
26. Start the server
27. What is FastAPI's automatic documentation?
28. What does FastAPI do for you?
29. FastAPI becomes especially interesting for AI
30. FastAPI + LLM

- 31. FastAPI + database
- 32. A more realistic API
- 33. What does {user\_id} mean?
- 34. Query parameters
- 35. Request body
- 36. Why Pydantic is important
- 37. The bigger picture
- 38. Why should you learn FastAPI?
- 39. What you should learn first
- 40. Your first FastAPI learning project
- 41. The one picture to remember

# 1. What is an API?

Imagine you go to a restaurant.

You sit at your table and look at the menu. You tell the waiter:

I want one masala dosa.

The waiter takes your request to the kitchen.

The kitchen prepares the dosa.

The waiter brings it back to you.

You don't need to know:

- how the kitchen works
- where the ingredients are stored
- how the dosa is prepared
- which cook prepared it
- how the kitchen communicates internally

You only need to know how to place the request and what you'll get back.

An API works in a very similar way.

API = Application Programming Interface

It is a defined way for one piece of software to ask another piece of software to do something or provide some information.

## 2. A real-world example

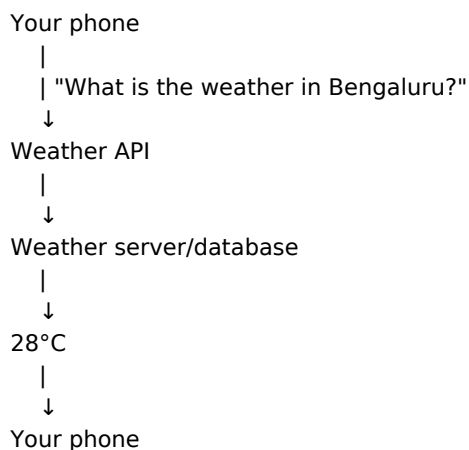
Suppose you use a weather application.

Bengaluru  
28°C

Where did that information come from?

The mobile application probably didn't measure the temperature itself.

Instead, something like this happens:



The API is the middleman that allows the applications to communicate.

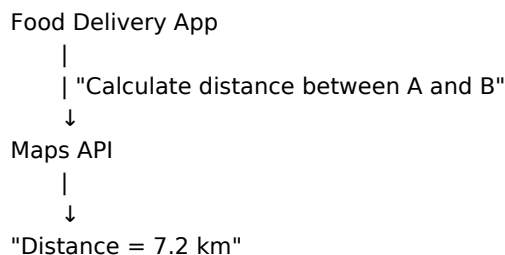
### 3. Another example: Google Maps

Suppose a food-delivery application wants to calculate the distance between:

Restaurant → Customer

The food-delivery company doesn't necessarily need to build its own complete mapping system.

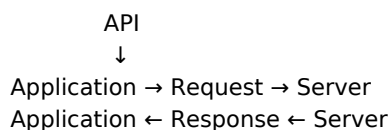
It can communicate with a mapping service through an API.



The food-delivery application receives the answer and displays it.

### 4. API in very simple words

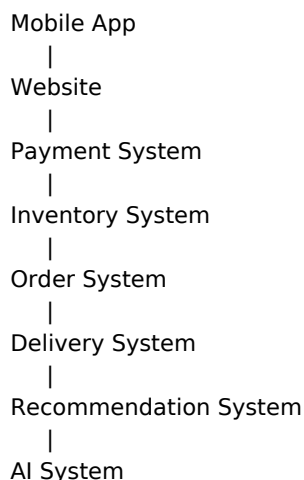
Think of an API as a waiter between two systems.



### 5. Why do we need APIs?

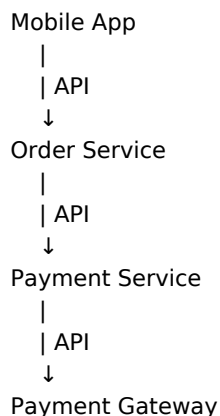
Without APIs, different applications would have difficulty communicating with each other in a controlled and standardized way.

Consider a company like Amazon.



These systems need to communicate.

APIs provide standardized doors through which they can communicate.



## 6. API is NOT a programming language

This is important.

API is not:

- Python
- Java
- JavaScript
- FastAPI
- C++
- SQL

API is a communication interface/convention.

You can build APIs using many technologies.

|            |               |
|------------|---------------|
| Python     | → FastAPI     |
| Python     | → Flask       |
| Python     | → Django      |
| Java       | → Spring Boot |
| JavaScript | → Express.js  |
| C#         | → ASP.NET     |
| Go         | → Gin         |

## 7. So what is FastAPI?

FastAPI is a Python framework for building APIs and web backends.

In simple words:

FastAPI gives Python developers the tools needed to create APIs easily.

Think of it this way:

```
graph TD;
  P[Python] -- "+" --> F[FastAPI];
  F -- "↓" --> B[API / Backend];
```

## 8. Python without FastAPI

Suppose you write:

```
def add_numbers(a, b):  
    return a + b
```

You can use it inside your Python program:

```
result = add_numbers(10, 20)  
  
print(result)
```

Output:

```
30
```

But there's a limitation.

Only your Python program can directly call that function.

What if you want:

```
Mobile App  
  ↓  
Internet  
  ↓  
Your Python program
```

The mobile app can't simply call:

```
add_numbers(10, 20)
```

because that function exists inside your Python program.

This is where FastAPI becomes useful.

## 9. Turning Python into an API

With FastAPI, we can expose that functionality through the internet/network.

```
from fastapi import FastAPI  
  
app = FastAPI()  
  
@app.get("/add")  
def add_numbers(a: int, b: int):  
    return {"result": a + b}
```

Now another application can make a request like:

```
GET /add?a=10&b=20
```

FastAPI calls:

```
add_numbers(10, 20)
```

and returns:

```
{"result": 30}
```

That's the basic idea.

## 10. Let's understand the code

```
from fastapi import FastAPI
```

We're importing FastAPI.

```
app = FastAPI()
```

We're creating our FastAPI application.

Think of this as:

"Create my web/API server."

```
@app.get("/add")
```

This is extremely important.

It tells FastAPI:

"When somebody sends a GET request to /add, run the function immediately below."

```
def add_numbers(a: int, b: int):
```

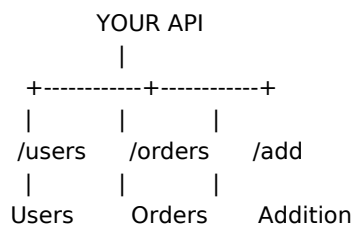
This is our Python function.

```
    return {"result": a + b}
```

returns the answer.

## 11. What is /add?

/add is called an endpoint or route.



Each endpoint provides a particular functionality.

```
GET /users
GET /users/123
POST /users
DELETE /users/123
GET /products
POST /orders
GET /orders/123
```

## 12. What is an endpoint?

An endpoint is basically:

A specific address through which another application can interact with your system.

```
https://example.com/users
```

Here:



`https://example.com`

is the server.

`/users`

is the endpoint.

## 13. GET, POST, PUT and DELETE

You will frequently hear these terms when learning APIs.

They represent different types of requests.

GET — Used to retrieve information.

`GET /products`

Meaning: "Give me the products."

POST — Used to create something.

`POST /users`

Meaning: "Create a new user."

PUT — Usually used to update something.

`PUT /users/123`

Meaning: "Update user 123."

DELETE — Used to delete something.

`DELETE /users/123`

Meaning: "Delete user 123."

## 14. A simple analogy

Think of an online shopping system.

| API                              | Meaning           |
|----------------------------------|-------------------|
| <code>GET /products</code>       | Show products     |
| <code>GET /products/10</code>    | Show product 10   |
| <code>POST /products</code>      | Create product    |
| <code>PUT /products/10</code>    | Update product 10 |
| <code>DELETE /products/10</code> | Delete product 10 |

You can think of these as instructions given to a shop employee.

GET  
"Show me."

POST  
"Create this."

```
PUT
"Change this."
```

```
DELETE
"Remove this."
```

## 15. What does an API return?

Most modern APIs commonly return JSON.

JSON is simply a structured way of representing information.

```
{
  "name": "Likhith",
  "age": 30,
  "city": "Bengaluru"
}
```

Think of JSON as a neatly organized digital form.

Instead of receiving:

```
Likhith 30 Bengaluru
```

you receive:

```
{
  "name": "Likhith",
  "age": 30,
  "city": "Bengaluru"
}
```

Now another program can easily understand it.

## 16. A complete FastAPI example

Let's build a tiny API.

Create:

```
main.py
```

Put this inside:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello from FastAPI!"}

@app.get("/add")
def add(a: int, b: int):
    return {
        "a": a,
        "b": b,
        "result": a + b
    }
```

We now have two endpoints.

#### Endpoint 1

GET /

returns:

```
{"message": "Hello from FastAPI!"}
```

#### Endpoint 2

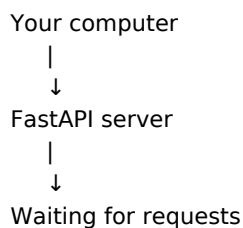
GET /add?a=10&b=20

returns:

```
{"a": 10, "b": 20, "result": 30}
```

## 17. How does this actually work?

When you start your FastAPI application:



Someone sends:

GET /add?a=10&b=20

FastAPI receives it.

It sees:

```
@app.get("/add")
```

So it knows:

"I need to execute the add() function."

It extracts:

```
a = 10  
b = 20
```

Then executes:

```
add(10, 20)
```

The function produces:

```
30
```

FastAPI converts the response into JSON:

```
{"a": 10, "b": 20, "result": 30}
```

and sends it back.

## 18. What is 127.0.0.1?

Don't let this scare you.

It basically means: This computer.

```
127.0.0.1:8000
```

means:

```
Your computer
+
Port 8000
```

You can also commonly see:

```
localhost:8000
```

which refers to your own computer.

## 19. What is port 8000?

Think of your computer as a large apartment building.

It has many doors.

```
Computer
|
├─ Door 80
├─ Door 443
├─ Door 3000
├─ Door 5000
└─ Door 8000
```

These are ports.

Your FastAPI server is listening on port:

```
8000
```

Therefore:

```
localhost:8000
```

means:

"Talk to the application listening on port 8000 on this computer."

## 20. Installing FastAPI

Now let's actually get started.

You should have Python installed.

```
python --version
```

or:

```
python3 --version
```

You should see something similar to:

## 21. Create a project

Create a folder:

```
fastapi-learning
```

Inside it:

```
fastapi-learning/  
└── main.py
```

## 22. Create a virtual environment

I strongly recommend doing this.

Inside your project:

```
python -m venv .venv
```

This creates:

```
fastapi-learning/  
├── .venv/  
└── main.py
```

A virtual environment is basically an isolated Python workspace for your project.

Imagine you have two kitchens.

```
Kitchen A  
└── Ingredients/tools for Project A  
  
Kitchen B  
└── Ingredients/tools for Project B
```

They don't interfere with each other.

That's approximately what a virtual environment does for Python packages.

## 23. Activate it

Windows

```
.venv\Scripts\activate
```

macOS/Linux

```
source .venv/bin/activate
```

Your terminal will usually show something like:

```
(.venv)
```

## 24. Install FastAPI

Run:

```
pip install "fastapi[standard]"
```

This installs FastAPI along with the standard tooling needed to run a development server.

## 25. Create your first application

main.py:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello FastAPI!"}
```

## 26. Start the server

Run:

```
fastapi dev main.py
```

You should see something similar to:

```
Uvicorn running on http://127.0.0.1:8000
```

Now your computer is running a web server.

Open:

```
http://127.0.0.1:8000
```

You should get:

```
{"message": "Hello FastAPI!"}
```

Congratulations.

You have created your first API.

## 27. What is FastAPI's automatic documentation?

This is one of the reasons FastAPI is so useful.

Once your server is running, go to:

```
http://127.0.0.1:8000/docs
```

FastAPI automatically creates an interactive API documentation page.

```
GET /
GET /add
POST /users
GET /users/{id}
```

You can even click an endpoint and test it.

This is extremely useful when developing APIs.

## 28. What does FastAPI do for you?

FastAPI isn't simply a way to write URLs.

It provides many useful features.

### 1. API routing

```
/users
/products
/orders
/login
/predict
/chat
```

### 2. Input validation

Suppose your API expects:

```
age: int
```

Someone sends:

```
age = "hello"
```

FastAPI can detect that the input isn't an integer and return an appropriate validation error.

### 3. Automatic documentation

FastAPI generates API documentation automatically.

### 4. JSON handling

FastAPI makes receiving and returning JSON straightforward.

### 5. Type hints

Python type hints such as:

```
name: str
age: int
price: float
```

help FastAPI understand what your API expects.

### 6. High performance

FastAPI is designed for high-performance web APIs and uses the ASGI ecosystem.

You don't need to understand ASGI yet.

For now, think:

FastAPI is designed to handle many API requests efficiently.

## 29. FastAPI becomes especially interesting for AI

This is particularly important if your goal is AI/ML/GenAI engineering.

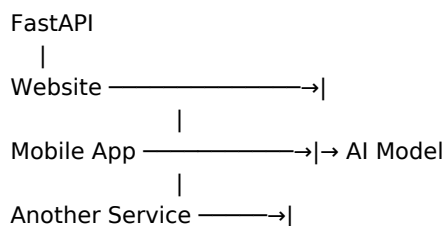
Imagine you've built a machine-learning model:

```
def predict_house_price(area, bedrooms):  
    ...
```

Your model works in Python.

But how does a website use it?

You can expose it using FastAPI.



For example:

```
@app.post("/predict")  
def predict(area: float, bedrooms: int):  
  
    price = model.predict(area, bedrooms)  
  
    return {  
        "predicted_price": price  
    }
```

Now other applications can call:

POST /predict

and receive:

```
{"predicted_price": 8500000}
```

This basic pattern is extremely important in modern AI applications.

## 30. FastAPI + LLM

Now let's move into Generative AI.

Suppose you've built an AI chatbot in Python.

You have:

```
def ask_ai(question):  
    answer = ...  
    return answer
```

A website can't directly call that Python function.

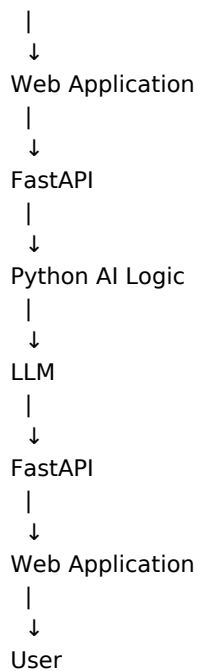
So you create:

POST /chat

Your architecture becomes:

User





For example:

```

from fastapi import FastAPI

app = FastAPI()

@app.post("/chat")
def chat(question: str):

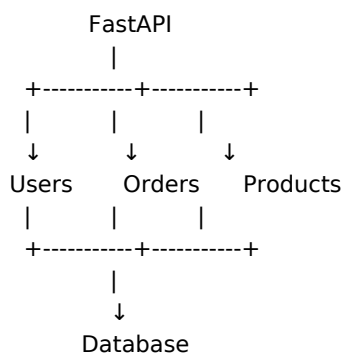
    answer = ask_ai(question)

    return {
        "question": question,
        "answer": answer
    }
  
```

This basic pattern is extremely important in modern AI applications.

## 31. FastAPI + database

You can also connect FastAPI to a database.



A request could be:

```
GET /users/123
```

FastAPI can:

- Receive the request.
- Validate the user ID.
- Query the database.
- Retrieve the user.
- Convert the result to JSON.
- Return it.

## 32. A more realistic API

Let's create a small user API.

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
users = [  
    {  
        "id": 1,  
        "name": "Rahul",  
        "age": 25  
    },  
    {  
        "id": 2,  
        "name": "Priya",  
        "age": 28  
    }  
]
```

```
@app.get("/users")  
def get_users():  
    return users
```

```
@app.get("/users/{user_id}")  
def get_user(user_id: int):  
  
    for user in users:  
  
        if user["id"] == user_id:  
            return user  
  
    return {  
        "message": "User not found"  
    }
```

Now:

```
GET /users
```

returns:

```
[  
    {
```

```
    "id": 1,
    "name": "Rahul",
    "age": 25
  },
  {
    "id": 2,
    "name": "Priya",
    "age": 28
  }
]
```

And:

```
GET /users/1
```

returns:

```
{
  "id": 1,
  "name": "Rahul",
  "age": 25
}
```

### 33. What does {user\_id} mean?

This:

```
@app.get("/users/{user_id}")
```

means the URL contains a variable.

For example:

```
/users/1
/users/2
/users/100
```

FastAPI takes the number and gives it to:

```
user_id
```

So:

```
/users/2
```

becomes:

```
user_id = 2
```

Then Python executes:

```
get_user(2)
```

### 34. Query parameters

Another common API pattern is:

```
/products?category=shoes&limit=10
```

Here:

```
category = shoes
limit = 10
```

are called query parameters.

Example:

```
@app.get("/products")
def get_products(category: str, limit: int = 10):

    return {
        "category": category,
        "limit": limit
    }
```

Request:

```
/products?category=shoes&limit=5
```

Response:

```
{
    "category": "shoes",
    "limit": 5
}
```

## 35. Request body

For POST requests, we often send a JSON body.

For example:

```
{
    "name": "Rahul",
    "age": 25
}
```

FastAPI can define exactly what it expects.

This is where Pydantic becomes important.

Example:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int

@app.post("/users")
def create_user(user: User):

    return {
        "message": "User created",
        "user": user
    }
```

Now FastAPI expects something like:

```
{
```

```
"name": "Rahul",  
"age": 25  
}
```

## 36. Why Pydantic is important

Imagine you're ordering food.

You tell the restaurant:

```
Name: Rahul  
Age: 25
```

But someone sends:

```
Name: Rahul  
Age: "banana"
```

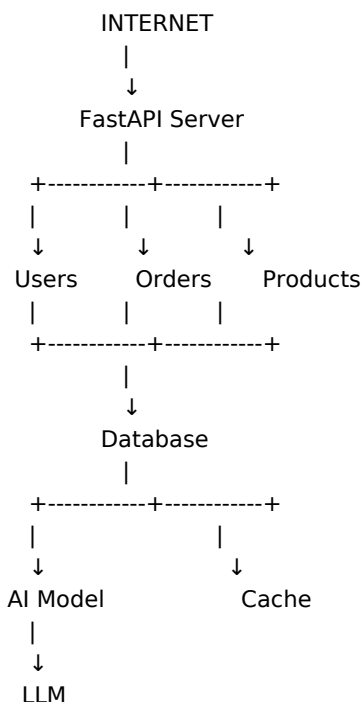
That doesn't make sense.

Pydantic helps define the expected structure and validate incoming data.

You can think of it as:

A security/checking officer that verifies incoming data before your Python code processes it.

## 37. The bigger picture



## 38. Why should you learn FastAPI?

For someone learning Python + AI/ML/GenAI, FastAPI is highly useful because it allows you to move from:

"I have a Python program"

to:

"I have a service that other applications can use."

That's a significant step.

For example:

#### Beginner Python project

```
Python script
↓
Reads file
↓
Processes data
↓
Prints result
```

#### Backend project

```
Client
↓
FastAPI
↓
Python
↓
Database
```

#### AI application

```
Frontend
↓
FastAPI
↓
Agent
↓
Tools
↓
Database / APIs
↓
LLM
```

This is much closer to how production AI applications are built.

## 39. What you should learn first

Don't try to learn all of FastAPI at once.

Follow this progression.

#### Level 1 — Python fundamentals

- Variables
- Data types
- if/else
- loops
- functions
- lists
- dictionaries
- classes
- exceptions

- modules
- virtual environments
- pip
- type hints

## Level 2 — Web/API fundamentals

- Client
- Server
- HTTP
- Request
- Response
- URL
- Endpoint
- GET
- POST
- PUT
- DELETE
- Status codes
- JSON
- Headers
- Query parameters
- Path parameters
- Request body

## Level 3 — FastAPI fundamentals

- FastAPI()
- @app.get()
- @app.post()
- Path parameters
- Query parameters
- Request bodies
- Pydantic
- Response models
- HTTP status codes
- Error handling
- Swagger/OpenAPI

## Level 4 — Real FastAPI

- Project structure
- Routers
- Dependencies
- Authentication
- Middleware
- Database
- SQLAlchemy
- Async programming
- Testing
- Environment variables
- Logging
- Docker
- Deployment

## Level 5 — AI + FastAPI

- FastAPI
- +
- LLMs
- +
- RAG

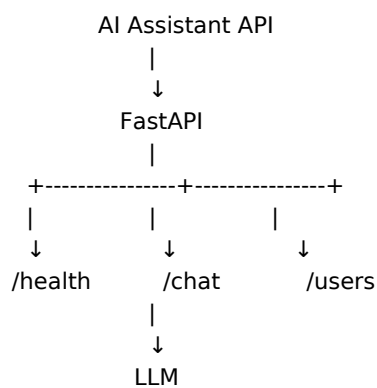
- + Vector databases
- + Agents
- + Tool calling
- + Streaming
- + Authentication
- + Observability

This is where FastAPI becomes particularly powerful for GenAI/Agentic AI applications.

## 40. Your first FastAPI learning project

I recommend that you don't just watch tutorials.

Build this progressively:



### Day 1

- FastAPI installation
- ↓
- Create main.py
- ↓
- Create /
- ↓
- Create /hello
- ↓
- Create /add
- ↓
- Run server
- ↓
- Use /docs

### Day 2

- GET
- POST
- Path parameters
- Query parameters
- JSON
- Pydantic

### Day 3



CRUD API  
Users  
Products  
Orders

## Day 4

Database  
SQLite  
SQLAlchemy

## Day 5

Authentication  
JWT  
Password hashing

## Day 6

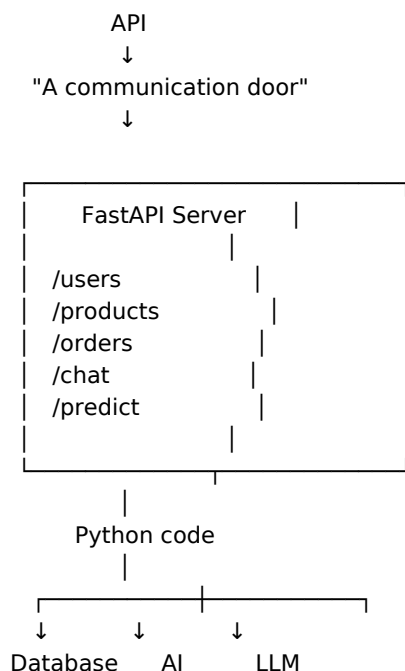
Connect an LLM  
/chat

## Day 7

Build a small AI API

# 41. The one picture I want you to remember

If you remember only one thing from this explanation, remember this:



API is the communication interface.

FastAPI is a Python framework that helps you build those interfaces.

Python contains the actual logic.

And together:

Python  
+

FastAPI  
+  
Database  
+  
AI/ML/LLM  
↓  
Production application

That's the foundation you need before moving into more advanced backend and Agentic AI development.